

Code Profiling & Performance Optimisation

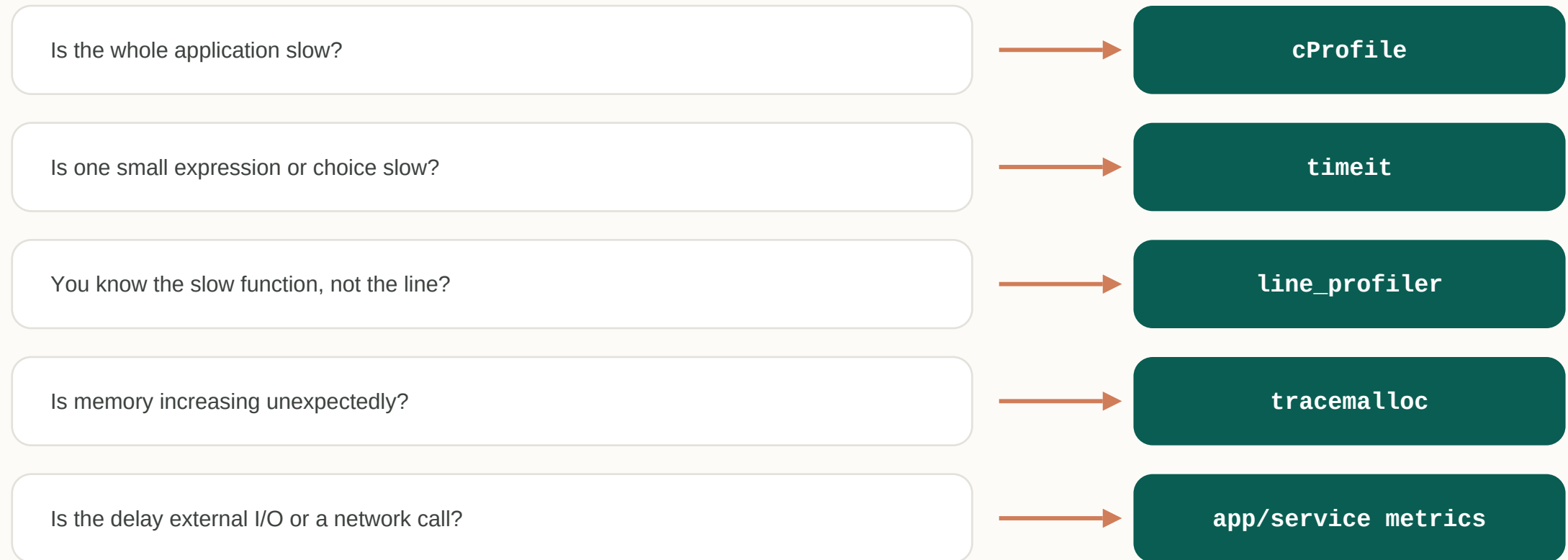
Learning Outcomes

- Explain what profiling actually measures, and recognise which situation calls for which tool.
- Run a profiler from a plain terminal, VS Code, or PyCharm, whichever a developer happens to have open.
- Use cProfile together with pstats to find exactly which function is consuming CPU time, and why.
- Use timeit to run small, controlled micro-benchmarks instead of guessing which approach is faster.
- Use tracemalloc to investigate where Python memory is actually being allocated, and how that changes over time.
- Use line-level profiling to drill down from “this function is slow” to “this exact statement is slow.”
- Interpret profiler output correctly - reading call counts and cumulative time together, not just chasing the single largest number on screen.
- Turn profiling evidence into one targeted optimisation, then re-measure to verify the change actually helped.

CONCEPTS

The Profiling Decision Tree

Different symptoms call for different tools - always name the actual question before reaching for one.



CONCEPTS

A Standard Project Layout

Keep the workload you want to measure completely separate from the code that measures it.

- `app.py` holds only the real application logic - nothing profiling-related lives inside it.
- `profile_app.py` imports that logic and drives it under a profiler, kept as its own file.
- `benchmarks.py` holds `timeit`-style micro-benchmarks for smaller, focused comparisons.
- This prevents profiling or benchmarking code from ever contaminating application logic that ships to production.



Example Workload: app.py

Save this as app.py. It deliberately contains repeated work, so a profiler has something meaningful to reveal.

- transform() is written as its own separate function, specifically so cProfile can count how many times it is called.
- process() calls transform() once per item, which is where the repetition comes from.
- Run it normally first with python app.py, before wrapping it in any profiler.
- Use this exact same workload for every profiling run in this deck, so results are comparable.
- Do not optimise this yet - the first objective is diagnosis, not a fix.

```
def transform(value):  
    return str(value).strip().upper()  
  
def process(values):  
    output = []  
    for value in values:  
        output.append(transform(value))  
    return output  
  
if __name__ == "__main__":  
    values = range(100_000)  
    result = process(values)  
    print("items:", len(result))
```

Running Profiling: Terminal, VS Code, PyCharm

The same underlying command works everywhere - only how you open a terminal changes.

- Terminal: run the program normally first, then reach for `-s cumulative` or `-s tottime` to change how results are sorted.
- VS Code: use the integrated terminal for repeatable commands, or a small profiling script for larger projects.
- PyCharm: has its own profiler UI, but running the same command in PyCharm's Terminal keeps the workflow portable.
- Whichever environment you use, keep the exact command documented in the project notes, so another developer can repeat it.
- On some systems the command is `python3` rather than `python` - check which one your project actually uses.



terminal

```
# Run it normally first
python app.py

# Profile it, unsorted
python -m cProfile app.py

# Sort by cumulative time (whole call paths)
python -m cProfile -s cumulative app.py

# Sort by function self-time
python -m cProfile -s tottime app.py

# Save the profile for later analysis
python -m cProfile -o profile.prof app.py
```

cProfile: The Simplest Application-Level Profiler

- Why use it: cProfile is built into the standard library, needs no installation, and profiles an entire program run in one command - it is the natural first step when you do not yet know where time is going.
- It measures function calls, not individual source lines, so its output is organised by function.
- -s cumulative sorts the report so the most expensive whole call paths appear first.
- Reach for this first whenever the question is simply “why is this program slow overall?”
- Its output can be redirected or saved to a file for later, more detailed analysis with pstats.



terminal

```
# No changes are required in app.py.  
# Terminal:  
python -m cProfile -s cumulative app.py
```

cProfile From Inside Python

Useful when you want profiling to cover one specific section of code, not the whole program.

- Why use it this way: wrapping only the workload you care about avoids polluting the report with import time or other setup noise.
- `enable()` starts collecting data; `disable()` stops it again - everything between the two is measured.
- `pstats.Stats()` turns the raw profile object into something you can sort, filter, and print.
- `print_stats(15)` shows only the top 15 entries, which is usually enough to spot the hotspot.
- This pattern is especially useful inside scripts, automated tests, and controlled one-off experiments.

```
profile_section.py
import cProfile
import pstats
from app import process

values = range(100_000)

profiler = cProfile.Profile()
profiler.enable()
process(values)
profiler.disable()

stats = pstats.Stats(profiler)
stats.sort_stats("cumulative")
stats.print_stats(15)
```


Interpreting cProfile's Columns

Every cProfile report uses the same six columns - learn what each one actually measures.

- **ncalls**: how many times the function was called. A recursive function shows two numbers, e.g. 3/1, meaning 3 total calls but only 1 was a fresh (primitive) call.
- **tottime**: time spent inside the function itself, excluding time spent inside functions it calls - this isolates the function's own work.
- **percall** (next to **tottime**): **tottime** divided by the number of primitive calls - the average self-time of one call.
- **cumtime**: time spent in the function plus everything it calls, all the way down - the cost of the whole call path.
- **percall** (next to **cumtime**): **cumtime** divided by primitive calls.
- **filename:lineno(function)**: exactly where the function is defined, so you can jump straight to the source.



cProfile output

ncalls	tottime	percall	cumtime	percall	filename:lineno(function)
100000	0.120	0.000001	0.450	0.000005	app.py:1(transform)
1	0.080	0.080000	0.530	0.530000	app.py:4(process)
1	0.010	0.010000	0.540	0.540000	app.py:11(<module>)

CONCEPTS

How to Read a Profiler Report, Step by Step

The exact numbers vary by machine - focus on the relationships between calls and timings, not the raw values.

- Step 1: transform has 100,000 calls - call frequency matters as much as per-call cost; a cheap operation repeated 100,000 times adds up.
- Step 2: process has low tottime (0.080) but high cumtime (0.530) - that gap of 0.450 is time spent inside functions it calls, mostly transform.
- Step 3: do not assume the function with the highest cumtime is automatically the root cause - here, process only looks expensive because of what it calls.
- Step 4: transform's own tottime (0.120) is the real self-time cost - this is the actual target for optimisation, not process.
- Step 5: always check callers and callees (see pstats, next) to confirm where time genuinely originates before changing any code.



reasoning

```
# Example interpretation only - numbers vary by machine:
#
# ncalls  tottime  percall  cumtime  percall  function
# 100000  0.120  0.000001  0.450  0.000005  transform
#      1  0.080  0.080000  0.530  0.530000  process
#      1  0.010  0.010000  0.540  0.540000  <module>
#
# Reasoning:
# high cumtime + low tottime -> look at what it CALLS
# high tottime                -> this function IS the work
```

pstats: Save and Inspect a Profile Offline

First save the profile to a file, then analyse it later without rerunning the application.

- Why use it: pstats lets you reopen a saved profile and re-slice it - sort differently, filter to one function, inspect callers or callees - without rerunning a potentially slow program each time.
- Create profile.prof once, with `python -m cProfile -o profile.prof app.py`.
- `sort_stats('cumulative')` highlights the call chains with the largest total cost.
- `print_callers()` shows every place that calls a given function - useful for finding who is generating excessive work.
- `print_callees()` shows what a function itself calls - useful for distinguishing a genuinely costly function from one that is merely called by a costly path.

```
inspect_profile.py
import pstats

stats = pstats.Stats("profile.prof")
stats.sort_stats("cumulative")
stats.print_stats(20)

print("\n--- CALLERS ---")
stats.print_callers("transform")

print("\n--- CALLEES ---")
stats.print_callees("process")
```

pstats: Filter and Focus the Investigation

Once a hotspot is known, reduce how much output you have to read through.

- Why filter: a full profile report on a real application can run to hundreds of lines - filtering keeps the investigation focused on what actually matters.
- Use `sort_stats('tottime')` when you specifically want functions that do substantial work themselves.
- Use `sort_stats('cumulative')` when investigating a complete call path rather than one function in isolation.
- `print_stats('transform')` restricts the printed report to entries matching that pattern.
- Remember: profile data is evidence - your job is still to explain why the evidence looks the way it does.

```
filter_profile.py
import pstats

stats = pstats.Stats("profile.prof")
stats.sort_stats("tottime")
stats.print_stats(10)

# Print only entries matching a pattern:
stats.print_stats("transform")

# Inspect callers/callees for one function:
stats.print_callers("transform")
stats.print_callees("transform")
```

timeit: Micro-Benchmark a Small Operation

Use timeit when the question is about one small operation, not an entire application.

- Why use it: timeit isolates a single expression from the noise of imports, I/O, and program warmup, then repeats it enough times that small differences become measurable.
- setup runs once, outside the timed statement - it prepares data without being counted in the result.
- number controls exactly how many times the statement executes.
- Always compare alternatives against identical inputs, or the comparison means nothing.
- Do not infer whole-application performance from a micro-benchmark alone - it only answers the narrow question it was built for.

```
benchmarks.py
import timeit

list_time = timeit.timeit(
    "value in values",
    setup="values = list(range(10000)); value = 9999",
    number=10000,
)
set_time = timeit.timeit(
    "value in values",
    setup="values = set(range(10000)); value = 9999",
    number=10000,
)

print("list:", list_time)
print("set :", set_time)
print("speedup:", list_time / set_time)
```

timeit From the Command Line

Convenient for quick, throwaway experiments in a VS Code terminal, PyCharm terminal, or a plain shell.

- Why use it this way: no script file needed at all - just a one-line question answered in seconds.
- -s supplies setup code that runs once and is not counted in the timing.
- Run each command independently to compare two competing expressions.
- timeit itself repeats each run and reports a stable-looking result, smoothing over one-off noise.
- Best suited to small language or data-structure questions, such as “is this faster in a list or a set?”



terminal

```
# Basic example:
python -m timeit "sum(range(100))"

# With setup:
python -m timeit -s "data=list(range(1000))" "sum(data)"

# Compare two expressions:
python -m timeit -s "data=list(range(1000))" "999 in data"
python -m timeit -s "data=set(range(1000))" "999 in data"
```

time.perf_counter(): Simple End-to-End Timing

Use perf_counter() when you need the elapsed time around a larger operation.

- Why use it: it is the simplest possible way to answer “how long did that block of code take, right now” - one number, no report to read.
- perf_counter() is specifically designed for measuring elapsed time, with the best resolution the system offers.
- Keep the timed region focused only on the operation being investigated.
- Do not include unrelated printing or setup inside the timed block unless that is genuinely part of what you want measured.
- For a robust comparison, repeat the measurement rather than trusting a single run.



```
time_workload.py
import time

start = time.perf_counter()
result = process(range(100_000))
elapsed = time.perf_counter() - start

print("result:", len(result))
print("elapsed:", elapsed, "seconds")
```

Line-Level Profiling With line_profiler

Install with: `python -m pip install line_profiler`. Best used once cProfile has already named a slow function.

- Why use it: cProfile tells you WHICH function is slow; line_profiler tells you WHICH STATEMENT inside that function is actually expensive.
- Mark the target function with an @profile decorator - this is injected automatically by the tool's runner, so no import is needed for it to work.
- Run it with kernprof, then view the line-by-line report with -v.
- Do not line-profile an entire large application - apply it only to the function already identified as hot.
- Interpret line time relative to hit count: a line executed 100,000 times can dominate the total even if each individual hit is cheap.

```
calculate_script.py
# Mark the target function:
@profile
def calculate(values):
    result = []
    for value in values:
        text = str(value)
        cleaned = text.strip().lower()
        result.append(cleaned[::-1])
    return result

# Run and view the line-by-line report:
# kernprof -l -v calculate_script.py
```


tracemalloc: Memory Allocation Profiling

Included with Python, and useful for investigating Python-level memory allocations specifically.

- Why use it: cProfile only measures CPU time and says nothing about memory - tracemalloc answers the completely different question of where memory is actually being allocated.
- start() begins allocation tracing for the current process.
- A snapshot captures the allocation state at one specific point in time.
- compare_to() reports the difference between two snapshots, which is usually far more useful than either snapshot alone.
- 'lineno' groups those differences by source line, pointing straight back to the code responsible.
- Remember: tracemalloc tracks Python-level memory allocations, not every native allocation made by C extensions.

```
memory_profile.py
import tracemalloc

tracemalloc.start()
before = tracemalloc.take_snapshot()

data = [str(i) * 10 for i in range(100_000)]

after = tracemalloc.take_snapshot()
stats = after.compare_to(before, "lineno")
for stat in stats[:10]:
    print(stat)

tracemalloc.stop()
```

tracemalloc: Identify the Biggest Allocation Sources

The output typically includes the file, line number, size difference, and allocation count.

- `size_diff` is the change in allocated bytes between the two snapshots - the headline number to look at first.
- `count_diff` is the change in the number of allocation blocks, which helps distinguish many small allocations from a few large ones.
- The traceback/line information points directly back to the source line responsible.
- Use this evidence to form a specific memory-retention or allocation hypothesis, the same discipline used for CPU hotspots.
- Always ask whether that memory is legitimately required by the workload, before assuming it is a leak.

```
memory_allocations.py
import tracemalloc

tracemalloc.start()
snapshot1 = tracemalloc.take_snapshot()

items = [{"id": i, "text": str(i)} for i in range(50_000)]

snapshot2 = tracemalloc.take_snapshot()
for stat in snapshot2.compare_to(snapshot1, "lineno")[:5]:
    print(
        "file:", stat.traceback,
        "size_diff:", stat.size_diff,
        "count_diff:", stat.count_diff,
    )
```

cProfile + tracemalloc: The Right Tool for the Question

CPU time and memory are different resources - a single profiler rarely explains both at once.

- Use cProfile for function-level CPU execution cost - “where is time going?”
- Use tracemalloc for Python memory allocation differences - “where is memory going?”
- Use both together when a single performance issue genuinely involves both CPU and memory, rather than guessing which one applies.
- Do not confuse high memory allocation with a memory leak - a workload can legitimately need a lot of memory without ever leaking any of it.

```
cpu_and_memory_profile.py

# CPU question:
# python -m cProfile -s cumulative app.py

# Memory question:
import tracemalloc

tracemalloc.start()
before = tracemalloc.take_snapshot()

# run workload here

after = tracemalloc.take_snapshot()
print(after.compare_to(before, "lineno")[:10])
tracemalloc.stop()
```

How to Interpret a Hotspot Correctly

- Ask how often the function actually runs - a high ncalls can matter more than a slow individual call.
- Ask how much time each individual call consumes on average, using percall.
- Distinguish self-time (tottime) from cumulative time (cumtime) before deciding what is genuinely expensive.
- Trace back to the caller that is actually creating the excessive work, rather than blaming the function on the receiving end.
- Check whether the workload used for profiling is representative of real usage - an unrealistic workload gives misleading numbers.
- Look for algorithmic causes (the wrong data structure, unnecessary repeated work) before reaching for low-level micro-optimisations.
- After making a change, confirm the same hotspot still matters - fixing one bottleneck often reveals the next one.

Example: Diagnosing Excessive Calls

A high call count is often a stronger clue than a slightly slow individual call.

- Running this under cProfile will show normalise() called 200,000 times - that number alone is worth investigating.
- Ask whether normalisation genuinely needs to happen once per item, or whether it could be cached, batched, or moved earlier.
- Do not start optimising until you understand why the calls are happening in the first place.
- Any proposed fix must preserve the existing output semantics - a faster function that returns the wrong answer is not an improvement.

```
app.py
def normalise(value):
    return str(value).strip().lower()

def process(values):
    result = []
    for value in values:
        result.append(normalise(value))
    return result

values = range(200_000)
result = process(values)
print(len(result))
```

Example Optimisation: Repeated Membership Testing

Profiling may reveal that list membership testing dominates a workload's runtime.

- List membership (`value in a_list`) is generally $O(n)$ - Python has to check items one by one.
- Set membership (`value in a_set`) is generally $O(1)$ on average - a direct hash lookup.
- Converting a list to a set has its own cost, so always benchmark the complete workload, not just the lookup in isolation.
- Before adopting the change, verify converting list to set has not altered required semantics such as ordering or duplicate handling.

```
app.py

def count_matches(values, targets):
    count = 0
    for value in values:
        if value in targets:
            count += 1
    return count

values = range(200_000)
targets = list(range(0, 200_000, 10))

# Baseline:
print(count_matches(values, targets))

# Candidate optimisation:
target_set = set(targets)

def count_matches_fast(values, target_set):
    return sum(1 for v in values if v in target_set)

print(count_matches_fast(values, target_set))
```

A Complete Before/After Experiment

Never report an optimisation without measuring both versions under the same conditions.

- Run multiple repetitions (here, 7) and take the minimum, rather than trusting a single noisy timing.
- Compare both versions against the exact same workload - values and targets are unchanged between runs.
- The assert statement checks correctness before any speed claim is made - a faster wrong answer is worthless.
- $\text{speedup} = \text{baseline} / \text{optimised}$: a result of 2.0 means the optimised version is roughly twice as fast under this measurement.

```
app.py

import time

def measure(fn, runs=7):
    times = []
    for _ in range(runs):
        start = time.perf_counter()
        result = fn()
        times.append(time.perf_counter() - start)
    return result, min(times)

baseline_result, baseline = measure(
    lambda: count_matches(values, targets)
)
fast_result, optimised = measure(
    lambda: count_matches_fast(values, target_set)
)

assert baseline_result == fast_result
print("baseline :", baseline)
print("optimised:", optimised)
print("speedup  :", baseline / optimised)
```

Common Profiling Mistakes

- Profiling a workload that is too small to expose the real bottleneck at all.
- Profiling setup code rather than the actual operation you meant to investigate.
- Changing the workload between the baseline run and the optimised run, making the comparison meaningless.
- Optimising whichever function has the largest name or number on screen, without understanding cumulative calls first.
- Treating one noisy timing as proof of anything, rather than repeating the measurement.
- Ignoring I/O, database, network, and other external-service latency, which no CPU or memory profiler will ever show you.
- Spending effort optimising code that contributes only a tiny percentage of the total runtime.

CONCEPTS

A Disciplined Profiling Workflow

1

Define the performance target: what does “fast enough” actually mean here?

2

Create a reproducible workload that represents real usage.

3

Measure the baseline before changing anything.

4

Profile with the smallest tool that can answer the question.

5

Identify the hotspot and explain why it is expensive.

6

Form one specific, testable optimisation hypothesis.

7

Implement exactly one targeted change.

8

Run correctness tests to confirm behaviour has not changed.

9

Re-measure using the exact same workload as the baseline.

10

Document the evidence, then keep or revert the change.

Tool Selection Cheat Sheet

cProfile

Application- and function-level CPU hotspots across a whole program run.

pstats

Inspects, sorts, and navigates a saved cProfile result without rerunning anything.

timeit

Small, controlled micro-benchmarks of one expression or statement.

perf_counter

The simplest possible elapsed-time measurement around a larger operation.

line_profiler

Statement-level investigation inside a function already known to be slow.

tracemalloc

Python memory allocation differences between two points in time.

IDE profilers

Convenient visual exploration - document the settings used so results stay reproducible.

Copy-and-Paste Profiling Template

Use this as the starting point for your own profiling experiments.

- Save this file as `profile_workload.py`, then run it directly with `python profile_workload.py`.
- Change only the body of `workload()` when experimenting - leave the profiling scaffolding around it untouched.
- Start with cumulative sorting, since it is the most useful default view of an unfamiliar program.
- Investigate the top few functions in the report first, rather than scanning the entire output line by line.

```
profile_workload.py

import cProfile
import pstats

def workload():
    # Replace this with the operation you want to investigate.
    values = range(100_000)
    return sum(x * x for x in values)

if __name__ == "__main__":
    profiler = cProfile.Profile()
    profiler.enable()
    result = workload()
    profiler.disable()

    stats = pstats.Stats(profiler)
    stats.sort_stats("cumulative")
    stats.print_stats(20)
    print("result:", result)
```

Practical Exercise: Profile → Hypothesise → Optimise

- Run the copy-and-paste template against a realistic workload of your own choosing.
- Record the top five entries from the cProfile report.
- Explain, in your own words, what ncalls, tottime, and cumtime suggest about each of those five entries.
- Select one hotspot and state a single, specific, testable hypothesis about why it is expensive.
- Make exactly one optimisation based on that hypothesis.
- Run the same workload again, unchanged, to keep the comparison fair.
- Report the baseline time, the optimised time, the resulting speedup, and confirmation of correctness.
- Repeat the cycle until the remaining bottleneck is outside the scope of this exercise.

Key Takeaways

- Profiling is measurement, not guesswork.
- cProfile is the best starting point for most Python application-level CPU investigations.
- pstats turns raw profile data into an explanation of how functions actually call one another.
- timeit is for controlled micro-benchmarks - not for judging a whole application.
- Line-level profiling earns its keep after a function has already been identified as hot, not before.
- tracemalloc investigates Python memory allocation behaviour, a genuinely different question from CPU time.
- Always compare like-for-like workloads, and always validate correctness before trusting a speed claim.
- The best optimisation is the one supported by evidence.